

HLHP V4 — Backend Integration Guide (Python)

What the interactive prototype (HLHP V4.dc.html / Light) expects from the backend: data contracts, the scoring engine to port, and the split between server data and client presentation.

1. Scope & architecture

The prototype is a 7-tab client (Today, Log, Streak, Recap, Patterns, Share, Learn). All numbers it shows are currently computed in-browser from mock zone weather. The backend replaces those mocks with real data; **all scoring stays deterministic on the server**. The AI layer (if used) only narrates numbers already computed — it never invents a score, pattern, or figure (see the numeral-validator rule in README_patterns.md).

weather provider → band mapping → SFI (env + personal) → /v2/today → client
user logs → gate check → pattern detection → /v2/patterns → client
↳ AI narration (story only, cached)

Existing reference code: hlhp_patterns_engine.py (gating + detection, stdlib-only, runnable demo), hlhp_patterns_prompts.py (narration prompts), patterns-implementation-spec-v1.md (build contract). This document covers everything the other six tabs need on top of that.

2. API surface

Suggested REST surface (Flask/FastAPI). All responses JSON; all dates ISO-8601; one profile per authenticated user.

Endpoint	Feeds tab	Returns
GET /v2/today	Today	weather, per-driver bands, env + personal SFI, mode, alert copy (L0/L1/tip), compound flag, scene key
POST /v2/logs	Log	accepts symptoms[] + areas[]; returns updated streak + log_days count
GET /v2/streak	Streak	current streak, last-7-days logged flags, next milestone (7/30)
GET /v2/recap? month=	Recap	30 daily entries {date, sfi null, dominant_driver}, event callouts, verdict vs previous month
GET /v2/patterns	Patterns	state machine payload from hlhp_patterns_engine.py (unchanged)
GET /v2/share	Share	week range dates, week avg, delta vs previous 7 days, 7 daily values, caption text
GET /v2/learn	Learn	explainers (evidence cells matched to logged symptoms × dominant driver), nuggets, nutrition/lifestyle levers

3. Scoring engine (port exactly)

3.1 Band tables — points per driver (0–25 each)

Driver	Bands → points
Temperature °C	<15 cold 8 · 15–19 cool 12 · 20–27 optimal 25 · 28–34 warm 12 · 35–42 hot 5 · >42 extreme 0
UV index	0–2 low 25 · 3–5 moderate 18 · 6–7 high 12 · 8–10 very_high 2 · 11+ extreme 0
Humidity %	<20 very_low 5 · 20–39 low 12 · 40–60 optimal 25 · 61–79 high 12 · 80–89 very_high 5 · ≥90 extreme 0
AQI	0–50 good 25 · 51–100 satisfactory 18 · 101–200 moderate 10 · 201–300 poor 5 · 301–400 very_poor 2 · >400 severe 0

Environmental SFI = sum of the four band points (0–100). **Dominant driver** = lowest band points (ties: Temperature → UV → Humidity → AQI order as listed).

3.2 Personal SFI (Personal mode headline)

```
CONCERN_WEIGHTS = {
  "Melasma": {"Temperature": 1, "AQI": 1, "UV": 2, "Humidity": 0.5},
  "Uneven Tone": {"Temperature": 1, "AQI": 1, "UV": 2, "Humidity": 0.5},
  "Acne": {"Temperature": 0.5, "AQI": 1.5, "UV": 0.5, "Humidity": 1.5},
  "Eczema": {"Temperature": 1, "AQI": 1, "UV": 0.5, "Humidity": 2},
} # default: all 1

def personal_sfi(bands: dict, concern: str, skin: str) -> int:
    wt = CONCERN_WEIGHTS.get(concern, DEFAULT_WEIGHTS)
    num = sum(wt[f] * bands[f].points for f in wt)
    den = 25 * sum(wt.values())
    base = 100 * num / den
    pen = sum(SKIN_BAND_PENALTY[f][bands[f].key].get(skin, 0) for f in bands)
    return max(0, min(100, round(base - pen)))
```

The SKIN_BAND_PENALTY matrix (per factor × band × skin type, anchor: Dry = −10 at very-low humidity) is in the prototype's logic class verbatim and in the v3.7 spec — copy it as data, do not re-derive. Guest mode: headline = environmental SFI, no profile deltas. Gender/age modifiers (v3.5 sheets) subtract risk_delta × 4 each, clamped to 0–100.

3.3 Mode ladder (thresholds on the headline SFI)

```
≥85 Paradise Mode · ≥70 Smooth Sailing · ≥55 Guard Up
≥40 Battle Stations · ≥25 Hostile Mode · <25 Code Red
```

The client animates a "mode sweep" through all six on entry — it only needs the final mode + SFI from the server; the sweep is presentational.

3.4 Scene key (weather scenery + ambient sound)

```
def scene(w, surge: bool) -> str:          # first match wins
    if surge:                             return "storm"
    if w.temp_c <= 8:                     return "snow"
    if w.wind_kmh >= 30:                  return "windy"
    if w.aqi >= 201:                      return "haze"          # polluted
    if w.temp_c >= 35:                    return "heat"
    if w.humidity >= 61:                  return "rain"
    return "clear"
```

Return this key in /v2/today; visuals and soundscapes per key live entirely on the client. Requires wind speed in the weather payload.

4. Payload examples

GET /v2/today

```
{
  "city": "Pune", "date": "2026-07-08", "mode_of_use": "personal",
  "weather": {"temp_c": 28, "uv_index": 6, "humidity_pct": 52,
             "aqi": 80, "wind_kmh": 10},
  "scene": "clear",
  "drivers": [
    {"factor": "Temperature", "band": "warm", "points": 12, "level": "Medium"},
    {"factor": "UV", "band": "high", "points": 12, "level": "Medium", "dominant": true},
    {"factor": "Humidity", "band": "optimal", "points": 25, "level": "Low"},
    {"factor": "AQI", "band": "satisfactory", "points": 18, "level": "Low"}
  ],
  "sfi": {"environmental": 67, "personal": 61, "headline": 61},
  "mode": "Guard Up",
  "alert": {"level": "L0", "l0": "...", "l1": "...", "tip": "...",
            "evidence": {"confidence": "HIGH", "pmids": ["..."]}},
  "compound": null, "surge": false
}
```

Levels for the impact bars derive from points: ≥20 Low, 10–19 Medium, <10 High. Alert copy comes from the evidence library cell for (dominant factor × band × skin × concern) — worst-first fallback exactly as in hlhp-interactive.html → lookupCell().

POST /v2/logs

```
{"date": "2026-07-08", "symptoms": ["breakout", "oily"],
 "areas": ["Forehead", "Chin"]}
→ 200 {"streak": 24, "log_days_30d": 26, "patterns_unlock_in": 4}
```

- Symptom vocabulary: normal · dry · oily · dull · breakout · spots. "normal" is exclusive — server should reject or clear other symptoms if it is present.
- Areas required only for breakout / spots; "Full face" is exclusive of specific zones.

- Max one log-day per calendar day (idempotent upsert); binge-logging must not raise counters.

5. Patterns tab

Use `hlhp_patterns_engine.py` as-is: unlock requires both gates in the same rolling 30-day window — ≥ 30 days since first log **and** ≥ 25 log-days. The client renders whatever state the engine returns (locked / progress / unlocked-active / decay etc.). Each pattern card needs: title, match % (0–100, drives the ring gauge), driver key (colours the card), chart series (timeline dots, week grid, or hourly bars), one-line mechanism, and one action CTA. AI narration is cached per (pattern, window) and passes the numeral validator before storage.

6. Server vs client responsibilities

Server (data)	Client (presentation — already built)
Weather + wind per city, band mapping, both SFIs, mode name, scene key, alert copy from evidence cells, streak/recap/share aggregates, pattern detection, caption text with real dates	Mode-sweep animation, calm→harsh ladder, orb colours, weather scenery + ambient soundscapes, all SFX (bells/chirps/applause), mascot moods, count-ups, confetti, share-card unfold

Rule of thumb: if it's a number, a band name, or a sentence — it comes from the server. If it moves or makes a sound — it's the client's job, keyed off those values.

7. Open items

- Wind speed is new in v4 (windy scene) — confirm the weather provider exposes it per city.
- Snow/chill scene triggers at ≤ 8 °C — only relevant for hill-station cities (Shimla in the prototype); confirm the city list.
- `SKIN_BAND_PENALTY` is marked DRAFT pending clinical sign-off — keep it as a data table so it can be swapped without a deploy.
- Share caption: server should localise dates (en-IN, "8 Jul") and keep the no-product-push voice from the Copy Bank.
- Surge (sudden-event) detection service is out of scope here — the prototype fakes it with a toggle; the client just needs surge: `true` + L2 copy.